

LOREBOOK

Modelagem de Dados e Definição Arquitetural

Proposta técnica complementar

Pedro Henrique de Araujo Pereira
Henrique Borges da Silva

2026

1. Objetivo do documento

Este documento apresenta a modelagem de dados e a organização arquitetural do LoreBook. A proposta busca manter o aplicativo simples de evoluir, com responsabilidades separadas entre interface, estado da aplicação, regras de domínio, acesso a dados e infraestrutura local.

A versão atual utiliza persistência local em SQLite e autenticação no próprio dispositivo. A estrutura foi definida para permitir evolução posterior para sincronização remota e integração de recursos de inteligência artificial sem concentrar essas responsabilidades nas telas.

2. Modelagem de dados

O modelo principal utiliza quatro entidades persistentes: usuários, obras, capítulos e sessão local.

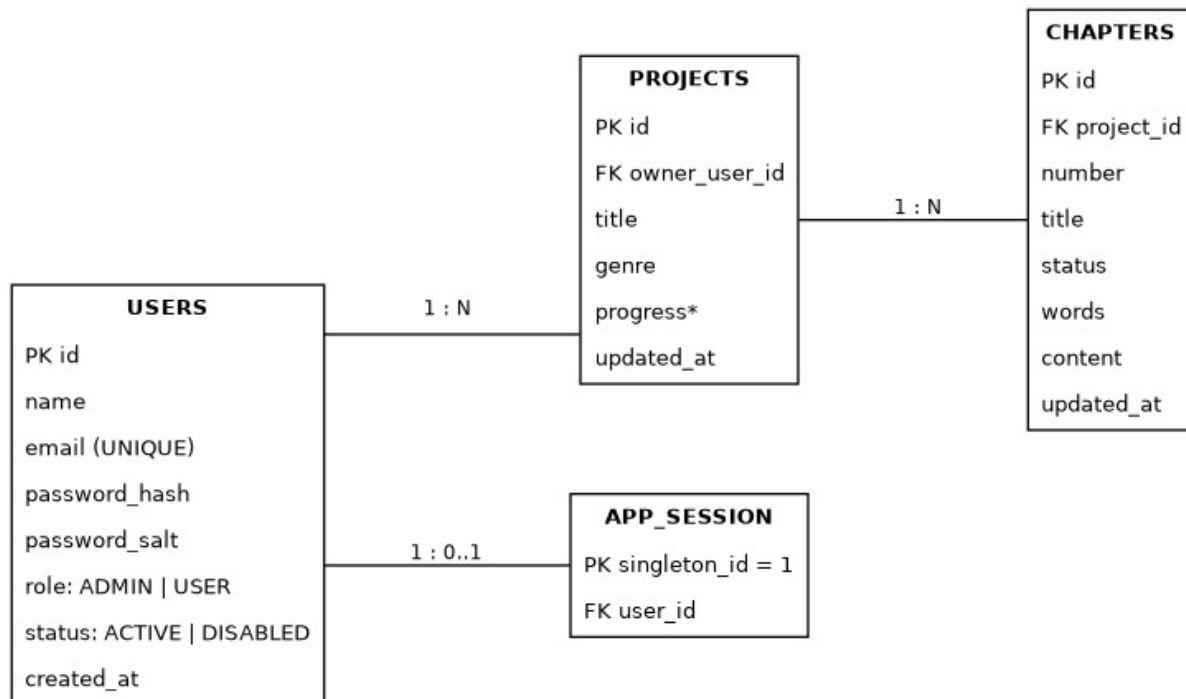


Figura 1 - Relacionamentos principais do banco local.

2.1. Relacionamentos

- Um usuário pode possuir várias obras; cada obra pertence a um único usuário.
- Uma obra pode possuir vários capítulos; cada capítulo pertence a uma única obra.
- O aplicativo mantém no máximo uma sessão local ativa, vinculada a um usuário.
- Capítulos herdam a autorização da obra, evitando a duplicação de user_id na tabela chapters.

2.2. Entidades e principais atributos

USERS

Campo	Tipo	Finalidade / restrição
id	TEXT	Chave primária.
name	TEXT	Nome do usuário.
email	TEXT	Único, sem diferenciação entre maiúsculas e minúsculas.
password_hash	TEXT	Hash da senha; a senha original não é persistida.
password_salt	TEXT	Salt individual usado no hash.
role	TEXT	ADMIN ou USER.
status	TEXT	ACTIVE ou DISABLED.
created_at	TEXT	Data de criação.

PROJECTS

Campo	Tipo	Finalidade / restrição
id	TEXT	Chave primária.
owner_user_id	TEXT	Chave estrangeira para users.id; define o proprietário.
title	TEXT	Título da obra.
genre	TEXT	Gênero informado pelo usuário.
progress	INTEGER	Campo mantido no schema; o progresso exibido é calculado pelos capítulos.
updated_at	TEXT	Última atualização.

CHAPTERS

Campo	Tipo	Finalidade / restrição
id	TEXT	Chave primária.
project_id	TEXT	Chave estrangeira para projects.id.
number	INTEGER	Número do capítulo dentro da obra.
title	TEXT	Título do capítulo.
status	TEXT	Rascunho, Em revisão ou Concluído.
words	INTEGER	Contagem de palavras do conteúdo.
content	TEXT	Texto do capítulo.
updated_at	TEXT	Última atualização.

APP_SESSION

Campo	Tipo	Finalidade / restrição
singleton_id	INTEGER	Chave primária fixa em 1, garantindo uma única sessão.
user_id	TEXT	Chave estrangeira para users.id.

2.3. Integridade e regras do banco

- O SQLite é inicializado com foreign_keys habilitado e journal_mode em WAL.
- E-mails são únicos, e os papéis e estados possuem valores restritos pelo schema.
- A combinação project_id + number é única em chapters, evitando números repetidos dentro da mesma obra.
- A exclusão de uma obra remove seus capítulos na mesma transação; o schema também possui ON DELETE CASCADE.

- Novas obras precisam de proprietário. Triggers impedem inserir ou atualizar uma obra deixando owner_user_id nulo.
- A sessão local possui uma única linha possível, identificada por singleton_id = 1.

3. Definição arquitetural

O LoreBook utiliza uma arquitetura em camadas com Repository Pattern. A separação busca impedir que telas executem SQL ou concentrem regras de autorização, mantendo cada parte do código com uma responsabilidade principal.

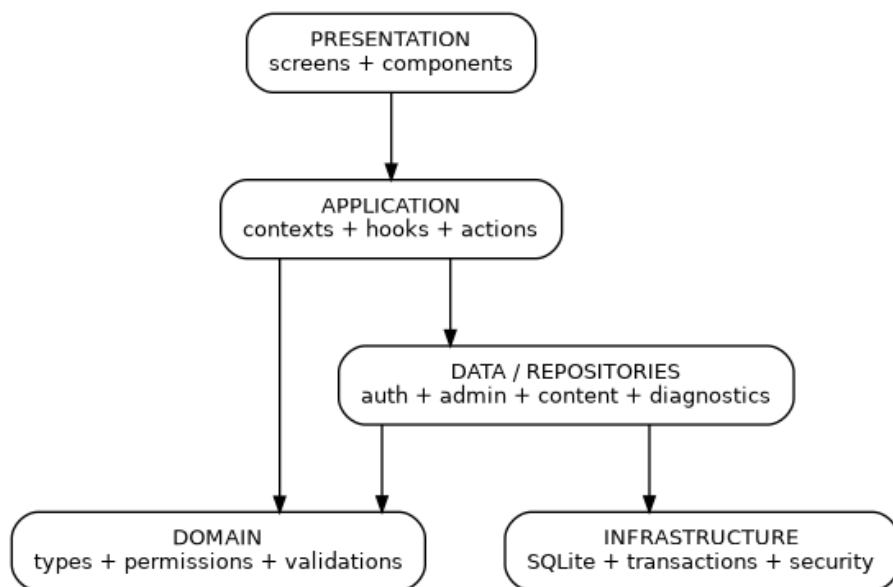


Figura 2 - Organização das camadas do aplicativo.

Camada	Responsabilidade	Localização
Presentation	Telas e componentes visuais. Recebe entradas do usuário e apresenta estados, erros, listas e formulários.	src/presentation/
Application	Coordena estado e fluxos por meio de contexts, hooks e actions.	src/application/
Domain	Tipos, validações e regras de autorização independentes de React e SQLite.	src/domain/
Data / Repositories	Centraliza consultas e comandos de usuários, obras, capítulos, administração e diagnóstico.	src/data/repositories/
Infrastructure	Implementa SQLite, transações e segurança local baseada em Expo Crypto.	src/infrastructure/

3.1. Repository Pattern

As telas não conhecem comandos SQL. Operações como autenticar usuário, listar obras, salvar capítulos e consultar dados administrativos passam por repositórios. Isso reduz o acoplamento com SQLite e facilita substituir ou complementar a origem dos dados no futuro.

3.2. Fluxos técnicos principais

Fluxo	Sequência
Autenticação	LoginScreen → AuthContext → authRepository → passwords/Expo Crypto + SQLite.
Acesso a uma obra	Tela → useProjectAccess/useAuthorization → regras de permission → contentRepository → SQLite.

Salvamento de capítulo	EditorScreen → useChapters/chapterActions → validação de domínio → contentRepository → transação SQLite.
Administração	AdminScreen → regras de ADMIN → adminRepository/diagnosticsRepository → SQLite.

4. Tecnologias e justificativas

Tecnologia / padrão	Justificativa
React Native + Expo	Permite construir a aplicação móvel em TypeScript e testar em Android, iOS e web durante o desenvolvimento.
TypeScript	Melhora a segurança de tipos e reduz erros em contratos entre telas, hooks, domínio e repositórios.
Expo SQLite	Fornece persistência local estruturada e suporte ao funcionamento sem conexão.
Expo Crypto	Utilizado para gerar salt aleatório e hash de senhas locais no protótipo.
React Context	Mantém autenticação e feedback compartilhados sem introduzir uma biblioteca de estado global adicional.
Repository Pattern	Isola o acesso a dados e prepara a aplicação para incorporar uma fonte remota posteriormente.

5. Evolução prevista

A arquitetura atual prioriza persistência local. Em uma evolução futura, os repositórios poderão coordenar o SQLite com uma API remota, permitindo sincronização entre dispositivos sem alterar a lógica central de propriedade de usuários, obras e capítulos.

Também está prevista a integração de recursos de inteligência artificial para apoio à escrita, revisão, organização de informações da obra e consulta contextual. Essa integração deverá ficar atrás de serviços da camada de aplicação e de uma API controlada, evitando chamadas diretas a provedores externos a partir das telas e mantendo a autoria e o controle dos dados com o usuário.

6. Síntese

O modelo atual utiliza usuários, obras, capítulos e sessão local como base persistente. A arquitetura em camadas com Repository Pattern mantém interface, regras e persistência separadas, atende às necessidades atuais do protótipo e cria uma base adequada para sincronização remota e recursos de IA em versões posteriores.